



EDUCAÇÃO A DISTÂNCIA

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

Unidade 1 | Semana 2

Sistemas de numeração e representação da informação

Unidades de representação



Com n bits, formamos 2^n padrões diferentes.



INSTITUTO FEDERAL
Sudeste de Minas Gerais

Educação a Distância
Pró-reitoria de Ensino

Curso Técnico em Desenvolvimento de Sistemas

Professor: Jeronimo Costa Penha

Apostila V1 | 2026



Semana 2 - Sistemas de numeração e representação da informação

Nesta semana, será estudado como números, caracteres e outras informações são representados em um sistema computacional. Para o usuário, um arquivo pode ser um texto, uma fotografia ou uma música. Para o hardware, entretanto, ele é uma sequência organizada de bits. As regras utilizadas para interpretar esses bits determinam o significado da informação.

O objetivo não é decorar sequências longas de zeros e uns. O estudante deverá compreender o funcionamento dos sistemas posicionais, dominar métodos verificáveis de conversão entre decimal, binário e hexadecimal e relacionar esses conhecimentos ao armazenamento e ao desenvolvimento de programas.

Delimitação da semana: Serão trabalhados principalmente números inteiros sem sinal. Aritmética binária, números negativos, complemento de dois, *carry* e *overflow* serão aprofundados na Semana 3.

Período de estudo: Carga estimada: 4 horas e 45 minutos, distribuídas entre leitura, exploração prática, revisão e questionário formativo.

Como utilizar a carga horária

A carga horária é semanal e pode ser distribuída conforme a rotina do estudante. Os tempos abaixo são estimativas para orientar o planejamento.

Etapa	Tempo	Finalidade
Leitura e anotações	2 h 25 min	Compreender os conceitos e acompanhar os exemplos resolvidos.
Prática guiada	1 h 10 min	Aplicar métodos de conversão, intervalos e capacidades.
Resumo e revisão	25 min	Retomar os conceitos essenciais e consultar o glossário.
Questionário formativo	30 min	Verificar a aprendizagem com feedback automático.
Fórum e dúvidas	15 min	Registrar dificuldades e revisar procedimentos.

Vídeos: Os vídeos que forem adicionados funcionarão como apoio ou substituição de parte do tempo de leitura, e não como carga extra obrigatória.

Objetivos de aprendizagem

- Explicar por que a representação binária é adequada aos circuitos digitais.
- Distinguir *bit*, *nibble*, *byte* e palavra.
- Interpretar números posicionais e os pesos associados a cada posição.

- Converter números entre os sistemas decimal, binário e hexadecimal.
- Calcular a quantidade de padrões e o intervalo sem sinal representável com n bits.
- Diferenciar unidades decimais de armazenamento de prefixos binários.
- Explicar, em nível introdutório, a relação entre ASCII, Unicode, UTF-8, bytes e bits.
- Reconhecer que o significado de um padrão binário depende do contexto e do tipo de dado.

Roteiro de estudos

1. Leia a apostila em pequenas seções e refaça os cálculos apresentados.
2. Monte uma tabela própria com as potências de 2 de 2^0 até 2^{16} .
3. Resolva a prática guiada antes de consultar as respostas comentadas.
4. Use o resumo, o glossário e o *checklist* para localizar pontos que precisam de revisão.
5. Realize o questionário formativo mais de uma vez e leia os *feedbacks*.
6. Utilize o Fórum de Dúvidas e Dificuldades quando um procedimento não estiver claro.

1. Informação, símbolos e representação

Informação é um conceito abstrato. Para armazená-la ou transmiti-la, precisamos representá-la por meio de símbolos organizados segundo regras. O número decimal 25, a palavra 'casa', uma fotografia e uma gravação de voz são informações diferentes, mas todas podem ser codificadas em padrões binários.

Uma representação estabelece uma correspondência entre algo que desejamos expressar e um conjunto de símbolos. No sistema decimal, utilizamos dez algarismos. No sistema binário, utilizamos apenas 0 e 1. Nas codificações de caracteres, valores numéricos são associados a letras e símbolos. Uma sequência simplificada do processo de transformação da informação em *bits* pode ser visto na Figura 1.

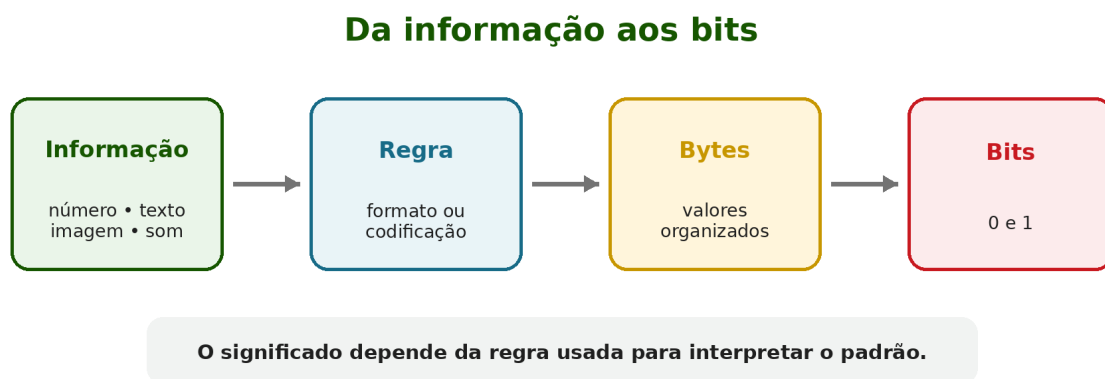


Figura 1 - A representação transforma informações em bytes e bits segundo regras definidas.

Ideia central: Bits não carregam um significado completo sozinhos. O tipo de dado, o formato do arquivo, a codificação e o programa determinam como eles serão interpretados.

1.1 Por que o computador utiliza binário?

Circuitos digitais trabalham de maneira confiável quando distinguem faixas de tensão associadas a dois estados lógicos. Esses estados podem ser representados por 0 e 1. A simplificação facilita a construção de portas lógicas, memórias, registradores e circuitos de comunicação.

Isso não significa que exista literalmente um pequeno número 0 ou 1 dentro do processador. O bit é uma abstração. Dependendo da tecnologia, os estados podem corresponder a níveis de tensão, cargas elétricas, orientações magnéticas ou outros fenômenos físicos.

Evite a simplificação excessiva: O estado 0 não significa obrigatoriamente ausência total de eletricidade, e o estado 1 não significa uma tensão universal. Os níveis físicos dependem da tecnologia e do circuito.

2. Bit, nibble, byte e palavra

O bit é a menor unidade lógica de representação binária e pode assumir dois valores. Quando vários bits são agrupados, o número de padrões possíveis cresce rapidamente. Com n bits, existem 2^n combinações distintas (Figura 2).

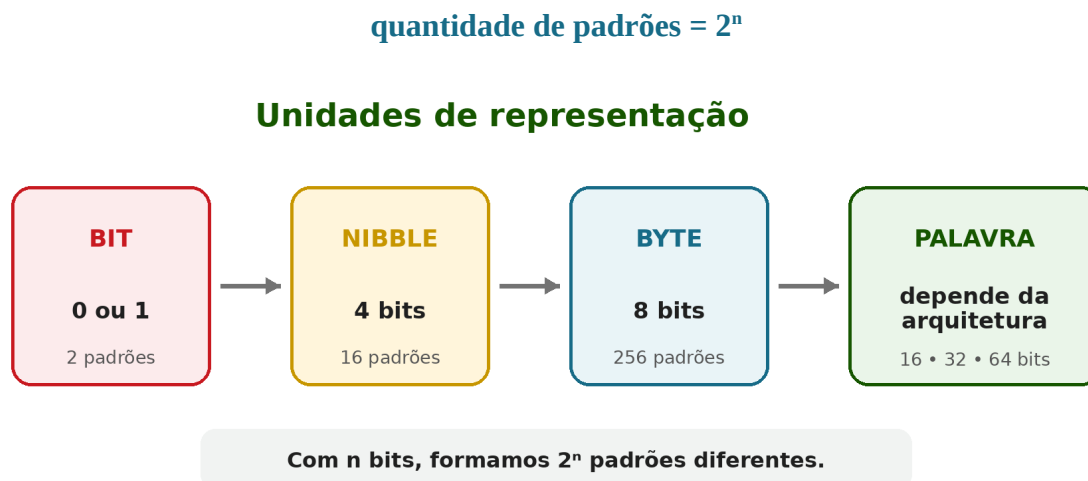


Figura 2 - Relação entre bit, nibble, byte e palavra.

- *Bit*: um dígito binário, 0 ou 1.
- *Nibble*: grupo de 4 bits. Corresponde exatamente a um algarismo hexadecimal.
- *Byte*: grupo de 8 bits. Pode formar 256 padrões diferentes.
- *Palavra*: quantidade de bits tratada como unidade natural por determinada arquitetura. Não possui tamanho universal.

2.1 Padrões e intervalos sem sinal

Quando os padrões binários representam números inteiros sem sinal, o primeiro padrão representa 0 e o último representa $2^n - 1$. O número de valores continua sendo 2^n , pois o zero também ocupa uma combinação.

Bits	Padrões	Intervalo sem sinal	Maior valor
1	2	0 a 1	1
4	16	0 a 15	15
8	256	0 a 255	255
12	4.096	0 a 4.095	4.095
16	65.536	0 a 65.535	65.535
32	4.294.967.296	0 a 4.294.967.295	4.294.967.295

EXEMPLO RESOLVIDO 1

Intervalo sem sinal com 12 bits

Determine quantos padrões podem ser formados e qual é o maior número inteiro sem sinal.

1. A quantidade de padrões é 2^{12} .
2. Como $2^{12} = 4.096$, existem 4.096 combinações.
3. O intervalo começa em 0. Portanto, o maior valor é $4.096 - 1 = 4.095$.

Resultado: Com 12 bits: 4.096 padrões e intervalo de 0 a 4.095.

3. Sistemas posicionais

Em um sistema posicional, o valor de um algarismo depende de sua posição. Cada posição possui um peso que é uma potência da base. A posição mais à direita dos inteiros possui expoente 0; os expoentes aumentam para a esquerda.

$$\text{valor} = \Sigma (\text{algarismo} \times \text{base}^{\text{posição}})$$

No número decimal 4.572, o algarismo 5 não representa simplesmente cinco unidades: por estar na posição das centenas, representa $5 \times 10^2 = 500$.

$$4572_{10} = 4 \times 10^3 + 5 \times 10^2 + 7 \times 10^1 + 2 \times 10^0$$

A mesma lógica é utilizada nas demais bases. O que muda é o conjunto de algarismos e os pesos posicionais.

3.1 Base e algarismos válidos

Sistema	Base	Algarismos válidos	Exemplos
Decimal	10	0, 1, 2, 3, 4, 5, 6, 7, 8, 9	2026_{10}
Binário	2	0, 1	11111101010_2
Hexadecimal	16	0 a 9, A, B, C, D, E, F	$7EA_{16}$ ou $0x7EA$

Um numeral é inválido se contiver um algarismo que não pertence à base. Por exemplo, 10201_2 não é binário porque contém o algarismo 2; $19A_{16}$ é válido, mas $19G_{16}$ não é, pois o hexadecimal termina em F.

3.2 Como indicar a base

Neste material, a base poderá aparecer como subscrito: 1010_2 , 10_{10} e A_{16} . Em linguagens de programação, a notação **0x** costuma indicar um valor hexadecimal, como 0x2A. Algumas linguagens utilizam **0b** para binário, como 0b101010. A forma exata depende da linguagem.

Atenção: Nunca presuma a base de uma sequência quando houver possibilidade de ambiguidade. O numeral 10 vale **dez** em decimal, **dois** em binário e **dezesesseis** em hexadecimal.

4. Potências de 2

As potências de 2 aparecem em conversões, tamanhos de memória, intervalos numéricos e organização de endereços. Não é necessário decorar uma tabela enorme, mas reconhecer as potências mais frequentes agiliza os cálculos.

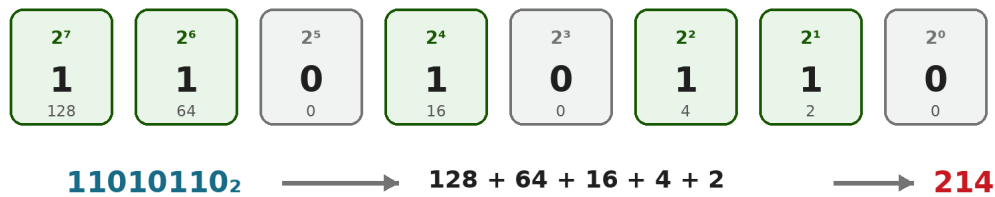
Potência	Valor	Potência	Valor	Potência	Valor
2^0	1	2^6	64	2^{12}	4.096
2^1	2	2^7	128	2^{13}	8.192
2^2	4	2^8	256	2^{14}	16.384
2^3	8	2^9	512	2^{15}	32.768
2^4	16	2^{10}	1.024	2^{16}	65.536
2^5	32	2^{11}	2.048	2^{20}	1.048.576

Estratégia: Cada potência é o dobro da anterior. Se você lembrar que $2^{10} = 1.024$, poderá obter 2^{11} , 2^{12} e outras por duplicação sucessiva.

5. Conversão de binário para decimal

Para converter um número binário para decimal, associe cada bit à potência de 2 correspondente. A posição mais à direita possui peso 2^0 . Depois, multiplique cada bit por seu peso e some os resultados (Figura 3).

O peso depende da posição



Somamos apenas os pesos das posições que contêm o bit 1.

Figura 3 - Conversão de 11010110_2 para decimal pelos pesos posicionais.

EXEMPLO RESOLVIDO 2

Converter 11010110_2 para decimal

Organize os pesos da direita para a esquerda e some apenas os pesos correspondentes aos bits 1.

1. Associe os bits aos pesos: 128, 64, 32, 16, 8, 4, 2 e 1.
2. Substitua os bits: $1 \times 128 + 1 \times 64 + 0 \times 32 + 1 \times 16 + 0 \times 8 + 1 \times 4 + 1 \times 2 + 0 \times 1$.
3. Some os termos diferentes de zero: $128 + 64 + 16 + 4 + 2 = 214$.

Resultado: $11010110_2 = 214_{10}$.

Conferência: 214 é menor que 256, portanto cabe em 8 bits sem sinal.

5.1 Método abreviado

Quando as potências já são conhecidas, pode-se escrever apenas os pesos associados aos bits 1. Entretanto, em atividades avaliativas, mostre os pesos ou a decomposição. Isso permite verificar o raciocínio e identificar um eventual erro de posição.

6. Conversão de decimal para binário

Um método geral para converter um inteiro decimal para binário consiste em dividir sucessivamente o valor por 2. Em cada divisão, registre o resto, que será 0 ou 1. Quando o quociente chegar a zero, leia os restos de baixo para cima (Figura 4).

Decimal → binário por divisões sucessivas

$173 \div 2 = 86$	resto 1
$86 \div 2 = 43$	resto 0
$43 \div 2 = 21$	resto 1
$21 \div 2 = 10$	resto 1
$10 \div 2 = 5$	resto 0
$5 \div 2 = 2$	resto 1
$2 \div 2 = 1$	resto 0
$1 \div 2 = 0$	resto 1

ler de baixo para cima

$173_{10} = 10101101_2$

Figura 4 - Conversão de 173_{10} para binário por divisões sucessivas.

EXEMPLO RESOLVIDO 3

Converter 173_{10} para binário

Faça divisões inteiras sucessivas por 2 e registre os restos.

- $173 \div 2 = 86$, resto 1; $86 \div 2 = 43$, resto 0.
- Continue até obter quociente 0: os restos são 1, 0, 1, 1, 0, 1, 0 e 1 na ordem em que foram calculados.
- Leia os restos na ordem inversa: 10101101.

Resultado: $173_{10} = 10101101_2$.

Conferência: $128 + 32 + 8 + 4 + 1 = 173$.

6.1 Conversão pela decomposição em potências

Também é possível localizar a maior potência de 2 que não ultrapassa o número e subtrair sucessivamente. Para 173, começamos em 128: $173 - 128 = 45$. Depois usamos 32, 8, 4 e 1. Os pesos usados recebem bit 1; os demais recebem bit 0.

$$173 = 128 + 32 + 8 + 4 + 1$$

$$173_{10} = 10101101_2$$

Qual método utilizar? Divisões sucessivas são sistemáticas e funcionam para qualquer inteiro positivo. A decomposição em potências costuma ser mais rápida quando o estudante domina as potências de 2.

7. Sistema hexadecimal

O hexadecimal utiliza base 16. Como precisamos de dezesseis símbolos, os valores de dez a quinze são representados pelas letras A, B, C, D, E e F. Maiúsculas e minúsculas normalmente possuem o mesmo valor numérico.

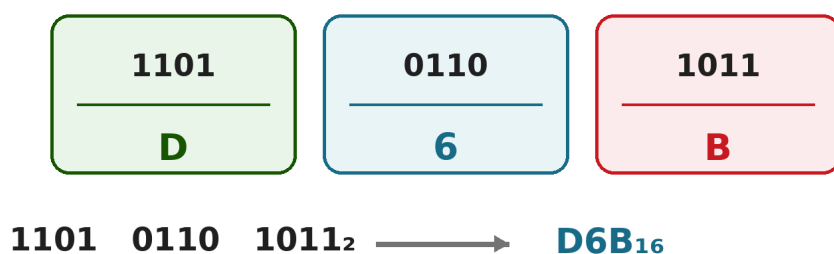
Hex	Dec	Bin	Hex	Dec	Bin
0	0	0000	8	8	1000
1	1	0001	9	9	1001
2	2	0010	A	10	1010
3	3	0011	B	11	1011
4	4	0100	C	12	1100
5	5	0101	D	13	1101
6	6	0110	E	14	1110
7	7	0111	F	15	1111

O hexadecimal é útil porque cada algarismo corresponde exatamente a quatro bits. Assim, uma sequência binária longa pode ser escrita de forma mais compacta sem perder a correspondência direta com os bits.

8. Conversão entre binário e hexadecimal

Para converter binário em hexadecimal, separe os bits em grupos de quatro começando pela direita. Se o grupo mais à esquerda estiver incompleto, acrescente zeros à esquerda. Depois substitua cada grupo pelo algarismo hexadecimal correspondente (Figura 5).

Binário ↔ hexadecimal: grupos de 4 bits



Se faltar um bit no grupo da esquerda, complete com zeros à esquerda.

Figura 5 - Conversão direta entre binário e hexadecimal.

EXEMPLO RESOLVIDO 4

Converter 110101101011_2 para hexadecimal

Separe o número em grupos de quatro bits.

1. Agrupe da direita para a esquerda: 1101 0110 1011.
2. Converta cada grupo: 1101 = D, 0110 = 6 e 1011 = B.
3. Junte os algarismos mantendo a ordem.

Resultado: $110101101011_2 = D6B_{16}$.

Conferência: $D6B_{16}$ retorna a 1101 0110 1011₂.

8.1 Hexadecimal para binário

Faça o caminho inverso: substitua cada algarismo hexadecimal por quatro bits. O valor $3A_{16}$, por exemplo, torna-se 0011 1010₂. Os zeros do primeiro grupo podem ser omitidos se a largura não estiver fixada, resultando em 111010₂.

Largura importa: Em registradores, endereços ou campos de instrução, os zeros à esquerda podem ser necessários para mostrar a largura definida, embora não alterem o valor numérico sem sinal.

9. Conversões envolvendo decimal e hexadecimal

Hexadecimal para decimal utiliza a decomposição posicional com potências de 16. Decimal para hexadecimal utiliza divisões sucessivas por 16. Restos de 10 a 15 devem ser escritos como A a F.

EXEMPLO RESOLVIDO 5

Converter $7EA_{16}$ para decimal

Associe os algarismos às potências de 16.

1. 7 está na posição 16^2 , E vale 14 e está na posição 16^1 ; A vale 10 e está em 16^0 .
2. Calcule: $7 \times 256 + 14 \times 16 + 10 \times 1$.
3. Some: $1.792 + 224 + 10 = 2.026$.

Resultado: $7EA_{16} = 2026_{10}$.

EXEMPLO RESOLVIDO 6

Converter 2026_{10} para hexadecimal

Divida sucessivamente por 16 e converta os restos acima de 9.

1. $2026 \div 16 = 126$, resto 10, que corresponde a A.
2. $126 \div 16 = 7$, resto 14, que corresponde a E.
3. $7 \div 16 = 0$, resto 7. Leia os restos de baixo para cima: 7EA.

Resultado: $2026_{10} = 7EA_{16}$.

Conferência: $7 \times 256 + 14 \times 16 + 10 = 2.026$.

10. Conferência e erros frequentes

Uma conversão deve produzir representações diferentes do mesmo valor. Sempre que possível, faça a operação inversa ou decomponha o resultado para conferir.

Erro frequente	Por que ocorre	Como evitar
Ler os restos de cima para baixo	Confusão no método de divisões sucessivas.	Ler do último resto para o primeiro.
Começar em 2^1	Esquecimento de que a posição direita é expoente zero.	Escrever 2^0 sob o bit mais à direita.
Agrupar binário pela esquerda	Perda da referência da posição menos significativa.	Agrupar sempre da direita para a esquerda.
Usar A como 11	Deslocamento na tabela hexadecimal.	Lembrar: A = 10 e F = 15.
Confundir padrões com maior valor	O zero também é representado.	Com n bits: 2^n padrões e máximo $2^n - 1$.

11. Unidades de armazenamento

Bits e bytes também são utilizados para expressar capacidade de memória, tamanho de arquivos e taxa de transmissão. É importante separar os prefixos decimais dos prefixos binários.

Unidade decimal	Bytes	Unidade binária	Bytes
1 kB	1.000	1 KiB	$1.024 = 2^{10}$
1 MB	1.000.000	1 MiB	$1.048.576 = 2^{20}$
1 GB	1.000.000.000	1 GiB	$1.073.741.824 = 2^{30}$
1 TB	1.000.000.000.000	1 TiB	$1.099.511.627.776 = 2^{40}$

Fabricantes de dispositivos de armazenamento normalmente informam capacidades utilizando prefixos decimais. Alguns sistemas operacionais calculam utilizando potências de 2, mas exibem rótulos que podem causar confusão. Por isso, um dispositivo anunciado como 500 GB pode aparecer com um número menor quando a unidade de comparação é binária.

Bit ou byte?: A letra b minúscula costuma indicar bit; B maiúscula indica byte. Assim, 100 Mb/s representa uma taxa em megabits por segundo, não 100 megabytes por segundo.

11.1 Estimando o tamanho de uma imagem

EXEMPLO RESOLVIDO 7

Imagem em tons de cinza sem compressão

Uma imagem possui 1.920×1.080 pixels e utiliza 8 bits por pixel. Desconsidere cabeçalhos e compressão.

1. Calcule os pixels: $1.920 \times 1.080 = 2.073.600$ pixels.
2. Como 8 bits equivalem a 1 byte, cada pixel ocupa 1 byte.
3. Tamanho: 2.073.600 bytes. Em MiB: $2.073.600 \div 1.048.576 \approx 1,98$ MiB.

Resultado: A imagem ocupa aproximadamente 2,07 MB ou 1,98 MiB sem compressão.

Arquivos reais podem ter cabeçalhos, metadados e compressão. Portanto, o cálculo fornece uma estimativa do conteúdo bruto, não necessariamente o tamanho final do arquivo.

12. Representação de caracteres

Textos também precisam ser convertidos em valores numéricos. Uma codificação de caracteres define relações entre símbolos e números. Sem uma regra comum, o mesmo valor poderia representar letras diferentes em programas distintos.

12.1 ASCII

O ASCII original utiliza 7 bits e define 128 códigos. Ele inclui letras do alfabeto inglês, algarismos, sinais de pontuação e caracteres de controle. Em sistemas atuais, esses códigos são normalmente armazenados em bytes.

Caractere	Decimal	Hexadecimal	Binário em 8 bits
A	65	41	01000001
a	97	61	01100001
0	48	30	00110000
espaço	32	20	00100000

Caractere	Decimal	Hexadecimal	Binário em 8 bits
LF (nova linha)	10	0A	00001010

Não memorize a tabela: O objetivo é compreender que caracteres podem ser associados a números. Quando necessário, tabelas e ferramentas podem ser consultadas.

12.2 Unicode e UTF-8

O ASCII não é suficiente para representar, de forma padronizada, os muitos sistemas de escrita, símbolos técnicos e emojis utilizados no mundo. O Unicode atribui pontos de código a caracteres, como U+0041 para A, U+00E7 para ç e U+20AC para €.

Um ponto de código é uma identificação abstrata. Para armazená-lo em bytes, utiliza-se uma forma de codificação. UTF-8 é uma codificação orientada a bytes e compatível com ASCII: os caracteres ASCII utilizam um byte; outros pontos de código utilizam sequências maiores (Figura 6).

Do caractere aos bytes em UTF-8

caractere	ponto de código	bytes (hex)	bits
A	U+0041	41	01000001
ç	U+00E7	C3 A7	11000011 10100111
€	U+20AC	E2 82 AC	11100010 10000010 10101100

Unicode identifica; UTF-8 define como armazenar em bytes.

Figura 6 - Exemplos de pontos de código Unicode e sua codificação em UTF-8.

Distinção importante: Unicode organiza e identifica caracteres; UTF-8 descreve como pontos de código são transformados em sequências de bytes.

13. O contexto define a interpretação

Um byte não possui um significado único. O padrão 01000001 representa 65 quando interpretado como inteiro sem sinal. Em uma codificação compatível com ASCII, representa a letra A. Em outro contexto, pode ser parte de uma cor, uma instrução ou um arquivo comprimido (Figura 7).

Os mesmos bits, interpretações diferentes

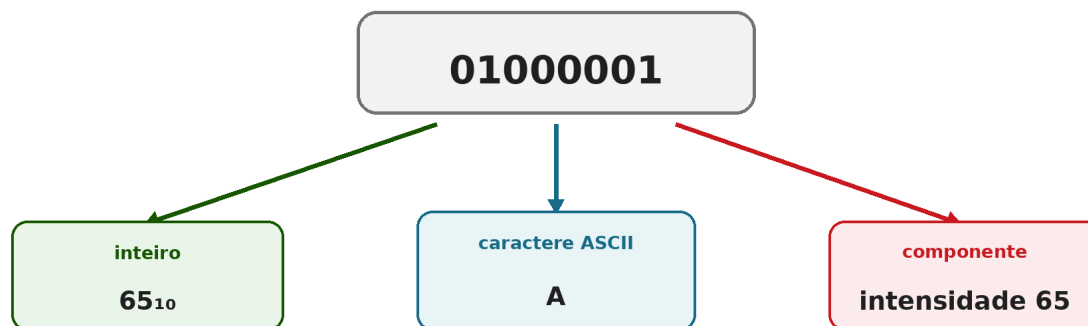


Figura 7 - Um mesmo padrão binário pode ser interpretado de maneiras diferentes.

Esse princípio ajuda a compreender a importância dos tipos de dados nas linguagens de programação e dos formatos de arquivo. Um programa precisa saber como agrupar os bytes e quais regras aplicar. Quando a regra escolhida está errada, os bits não mudam, mas o resultado apresentado pode não fazer sentido.

Exemplo cotidiano: Quando um arquivo de texto é aberto com a codificação incorreta, podem aparecer símbolos estranhos. O problema não está necessariamente nos bytes, mas na interpretação usada para convertê-los em caracteres.

14. Exemplo integrador

Considere o byte 00111010_2 .

1. Como inteiro sem sinal: $32 + 16 + 8 + 2 = 58_{10}$.
2. Em hexadecimal: $0011 = 3$ e $1010 = A$, portanto $3A_{16}$.
3. Em ASCII: o valor decimal 58 corresponde ao caractere dois-pontos (:).
4. Como parte de outro formato, o mesmo byte pode ter uma interpretação completamente diferente.

$$00111010_2 = 58_{10} = 3A_{16} = ':' \text{ em ASCII}$$

A igualdade acima compara representações do mesmo valor. O caractere somente aparece porque escolhemos interpretá-lo segundo ASCII. Essa relação entre bits, valor e significado será retomada ao estudarmos instruções, registradores e memória.

15. Prática guiada

Resolva os exercícios em papel. Registre os pesos, divisões ou agrupamentos utilizados. Consulte as respostas somente depois de concluir sua tentativa.

1. Converta 10011101_2 para decimal.
2. Converta 245_{10} para binário.
3. Converta 111011101101_2 para hexadecimal.
4. Converta $4F7_{16}$ para binário e decimal.
5. Converta 4095_{10} para hexadecimal.
6. Quantos padrões podem ser formados com 20 bits? Qual é o maior inteiro sem sinal?
7. Um sensor registra 16 bits por amostra e produz 48.000 amostras por segundo. Quantos bytes são produzidos em um segundo, desconsiderando cabeçalhos e compressão?
8. Uma imagem RGB possui 800×600 pixels e utiliza 24 bits por pixel. Estime o tamanho bruto em bytes e em MiB.
9. Explique a diferença entre 1 MB e 1 MiB.
10. Explique, com suas palavras, a diferença entre ponto de código Unicode e UTF-8.

15.1 Respostas comentadas

1. $10011101_2 = 128 + 16 + 8 + 4 + 1 = 157_{10}$.
2. $245 = 128 + 64 + 32 + 16 + 4 + 1$; portanto, $245_{10} = 11110101_2$.
3. $1110\ 1110\ 1101_2 = EED_{16}$.
4. $4F7_{16} = 0100\ 1111\ 0111_2$. Em decimal: $4 \times 256 + 15 \times 16 + 7 = 1.271_{10}$.
5. $4095_{10} = FFF_{16}$, pois $4095 = 16^3 - 1$ e três algarismos F representam doze bits iguais a 1.
6. $2^{20} = 1.048.576$ padrões. O maior valor sem sinal é 1.048.575.
7. 16 bits = 2 bytes por amostra. $48.000 \times 2 = 96.000$ bytes por segundo.
8. $800 \times 600 \times 24 = 11.520.000$ bits. Dividindo por 8: 1.440.000 bytes. Em MiB: $1.440.000 \div 1.048.576 \approx 1,37$ MiB.
9. MB utiliza potência de 10: 1.000.000 bytes. MiB utiliza potência de 2: 1.048.576 bytes.
10. O ponto de código identifica abstratamente um caractere; UTF-8 define a sequência de bytes usada para codificá-lo.

16. Atividade de exploração

Escolha um arquivo de texto simples que contenha letras sem acento, letras acentuadas e pelo menos um símbolo ou emoji. Observe o tamanho do arquivo e responda em suas anotações:

- A quantidade de bytes é sempre igual à quantidade visual de caracteres?
- Quais caracteres provavelmente ocupam mais de um byte em UTF-8?
- O que acontece com o tamanho se o mesmo texto for repetido muitas vezes?
- Por que o tamanho pode incluir bytes adicionais, como marcações de quebra de linha?

A atividade é destinada à observação e à fixação. Não é necessário enviar uma resposta, salvo orientação posterior do professor.

17. Resumo da semana

- Computadores armazenam padrões de bits; o significado depende das regras de interpretação.
- Com n bits, existem 2^n padrões; para inteiros sem sinal, o intervalo é de 0 a $2^n - 1$.
- Em sistemas posicionais, cada posição possui peso igual a uma potência da base.
- Binário utiliza base 2; decimal, base 10; hexadecimal, base 16.
- Binário para decimal usa decomposição em potências de 2.
- Decimal para binário pode usar divisões sucessivas por 2.
- Cada algarismo hexadecimal corresponde a quatro bits.
- Prefixos decimais e binários de armazenamento representam quantidades diferentes.
- ASCII associa códigos a um conjunto limitado de caracteres.
- Unicode identifica caracteres; UTF-8 codifica pontos de código em bytes.
- Aritmética binária e complemento de dois serão estudados na Semana 3.

18. Glossário

Termo	Definição
Base	Quantidade de algarismos disponíveis em um sistema posicional.
Bit	Dígito binário que assume o valor 0 ou 1.
Byte	Grupo de 8 bits.
Nibble	Grupo de 4 bits, equivalente a um algarismo hexadecimal.
Palavra	Unidade natural de dados de uma arquitetura; seu tamanho depende do sistema.
MSB	Bit mais significativo, localizado à esquerda em uma representação convencional.
LSB	Bit menos significativo, localizado à direita e associado a 2^0 .
Hexadecimal	Sistema posicional de base 16, com algarismos de 0 a F.
Código de caractere	Regra que associa caracteres a valores numéricos.
Ponto de código	Identificador numérico abstrato atribuído a um caractere no Unicode.
UTF-8	Forma de codificação Unicode orientada a bytes e compatível com ASCII.

19. Checklist antes do questionário

- Consigo identificar a base e os algarismos válidos de um numeral.
- Consigo explicar os pesos posicionais sem começar indevidamente em expoente 1.
- Consigo converter binário e hexadecimal para decimal mostrando os cálculos.
- Consigo converter decimal para binário ou hexadecimal por divisões sucessivas.

- Consigo agrupar bits em conjuntos de quatro da direita para a esquerda.
- Consigo calcular 2^n e o maior valor sem sinal com n bits.
- Consigo distinguir bit, byte, nibble e palavra.
- Consigo explicar a diferença entre MB e MiB.
- Consigo explicar a relação entre ASCII, Unicode, UTF-8, bytes e bits.
- Consigo reconhecer que o mesmo padrão de bits pode ter significados diferentes.

Próxima semana: Estudaremos soma e subtração binárias, números com sinal, complemento de dois, carry, overflow e sua relação com a ULA, registradores e flags do processador.

Referências

PATTERSON, David A.; HENNESSY, John L. Computer Organization and Design: The Hardware/Software Interface. 5. ed. Waltham: Morgan Kaufmann, 2014.

THE UNICODE CONSORTIUM. The Unicode Standard, Version 17.0. Disponível em: <https://www.unicode.org/versions/Unicode17.0.0/>. Acesso em: 1 set. 2026.

THE UNICODE CONSORTIUM. FAQ: UTF-8, UTF-16, UTF-32 & BOM. Disponível em: https://unicode.org/faq/utf_bom.html. Acesso em: 1 set. 2026.

As figuras desta apostila foram elaboradas especificamente para este material didático.